

REPORT 14 - Spiral Model Research

Levi Nelson, Carlos W. Mercado

Brigham Young University-Idaho

CSE 272 - Software Lifecycle Models

Instructor: Br. Edward Bullen

Date: 04/09/2025

<i>Report - Spiral Model Research - Week 14</i>	2
1. OVERVIEW	5
1.1 Spiral (Software Process) Model Overview	5
2. MEETINGS	10
What meetings does this methodology require?	10
2.1 Product Requirements Meeting	10
2.1.1 Attendees	10
2.1.2 Agenda	10
2.1.3 Purpose	10
2.1.4 Event	11
2.2 Phase Planning Meeting	11
2.2.1 Attendees	11
2.2.2 Agenda	11
2.2.3 Purpose	11
2.2.4 Event	11
2.3 Phase Review Meeting	11
2.2.1 Attendees	11
2.2.2 Agenda	12
2.2.3 Purpose	12
2.2.4 Event	12
2.3 Phase Retrospective Meeting	12
2.3.1 Attendees	12
2.3.2 Agenda	12
2.3.3 Purpose	13
2.3.4 Event	13
2.4 Weekly Stand-up	13
2.4.1 Attendees	13
2.4.2 Agenda	13
2.4.3 Purpose	13
2.4.4 Event	14
3. DOCUMENTS	15
3.1 Requirements & Lifecycle Plan	15
3.1.1 Author	15
3.1.2 Intended Audience	15
3.1.3 Purpose	15
3.1.4 Deadline	15
3.2 Development Plan	16
3.2.1 Author	16
3.2.2 Intended Audience	16
3.2.3 Purpose	16
3.2.4 Deadline	16
3.3 Integration Plan	16

3.3.1 Author	16
3.3.2 Intended Audience	17
3.3.3 Purpose	17
3.3.4 Deadline	17
3.4 Integration Plan	17
3.4.1 Author	17
3.4.2 Intended Audience	17
3.4.3 Purpose	17
3.4.4 Deadline	18
3.5 Test Plan	18
3.5.1 Author	18
3.5.2 Intended Audience	18
3.5.3 Purpose	18
3.5.4 Deadline	18
4. ROLES	19
4.1 Project Manager	19
4.1.1 Qualifications	19
4.1.2 Responsibilities	19
4.2 Software Engineer	19
4.2.1 Qualifications	19
4.2.2 Responsibilities	19
4.3 Software Tester	20
4.3.1 Qualifications	20
4.3.2 Responsibilities	20
5. CHECKPOINTS	21
What milestones and/or checkpoints does this methodology require?	21
5.0 Checkpoint 0	21
5.0.1 What is the name of the checkpoint?	21
5.0.2 How does one know that the checkpoint is reached?	21
5.1 Checkpoint 1 - LCO	22
5.1.1 What is the name of the checkpoint?	22
5.1.2 How does one know that the checkpoint is reached?	22
5.2 Checkpoint 2 - LCA	23
5.2.1 What is the name of the checkpoint?	23
5.2.2 How does one know that the checkpoint is reached?	24
5.3 Checkpoint 3 - IOC	25
5.3.1 What is the name of the checkpoint?	25
5.3.2 How does one know that the checkpoint is reached?	25
7. REFERENCES	26
7.1 Primary Sources	26
7.2 Secondary Sources	26

1. OVERVIEW

1.1 Spiral (Software Process) Model Overview

The Spiral Model was presented in 1987 by Barry W. Boehm as an alternative to multiple software development models used at the time¹. Following this purpose, the author stated the following premises (Boehm, 1988):

1. Distinguishing feature: “The Spiral Model presented in this article is one candidate for improving the software process model situation. The major distinguishing feature of the Spiral Model is that it creates a *risk-driven* approach to the software process rather than a primarily *document-driven* or *code-driven* process”.
2. What this model it is not: “[A] process model differs from a software method (often called a methodology) in that a method’s primary focus is on how to navigate through each phase (determining data, control, or ‘uses’ hierarchies; partitioning functions; allocating requirements) and how to represent phase products (structure charts; stimulus-response threads; state transition diagrams).
3. What this model is: “[This model] provide[s] guidance on the order (phases, increments, prototypes, validation tasks, etc.) in which a project should carry out its major tasks [unlike what software methodologies perform].”

¹ Code-and-fix Model, Stagewise Model and Waterfall Model, Evolutionary Development Model, and Transform Model.

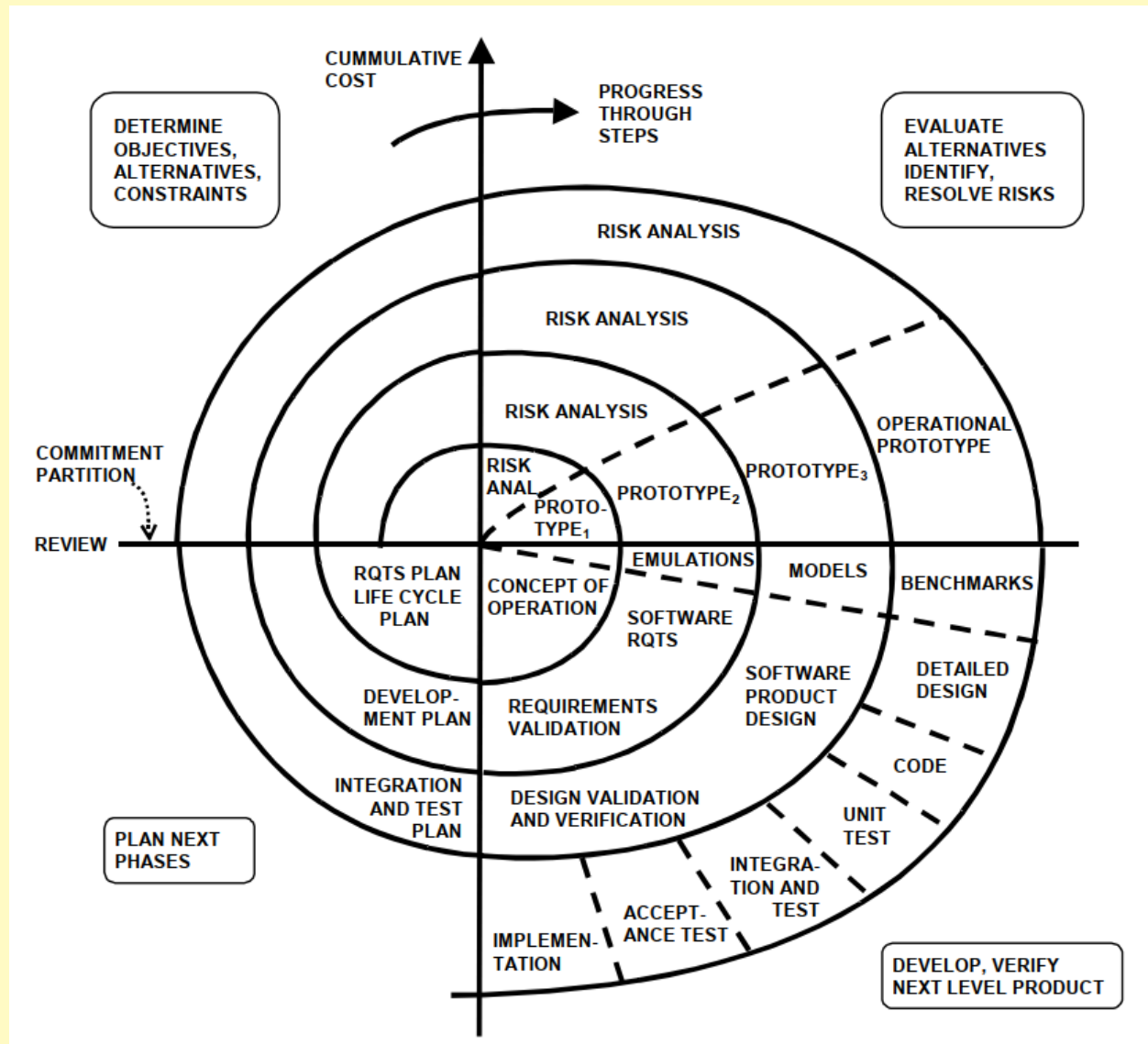


Figure 1 - Spiral model of the software process, original diagram (Boehm, 1988) as redrawn by Barry W. Boehm (2000).

Thus, this software process model is described by Boehm (1988) as “based on experience with various refinements of the waterfall model”. The major Spiral Model features are as follows (Boehm, 2000):

- Cyclic concurrent engineering;
- Risk driven determination of process and product;
- Growing a system via risk-driven experimentation and elaboration;

- And lowering development cost by early elimination of nonviable alternatives and rework avoidance.

As a result of planning and risk analysis, different projects may choose different processes. That is, the Spiral Model is actually a **risk-driven process model generator**, in which different risk patterns can lead to choosing incremental, waterfall, evolutionary prototyping, or other subsets of the process elements in the Spiral Model diagram.

Figure 1 depicts the conceptualization of the model. For understanding it, we will follow the explanation provided by the author (Boehm, 1988):

1. The **radial dimension** in *Figure 1* represents the **cumulative cost** incurred in accomplishing the steps to date. The more we move away from the origin over the vertical axis, the higher are the costs.
2. The **angular dimension** represents the **progress made** in completing each cycle of the spiral. The horizontal axis is the review stage at every intersection.
3. The model reflects the underlying concept that **each cycle involves a progression that addresses the same sequence of steps**, for each portion of the product and for each of its levels of elaboration, from an overall concept of operation document down to the coding of each individual program.

The model is characterized by a set of six “success-critical invariant attributes”. These are six properties which EVERY spiral development process MUST incorporate (Boehm, 2000). Let’s list them and characterize them:

→ *Spiral Invariant 1* - *Concurrent Determination of Key Artifacts*

- ◆ Concurrent rather than sequential determination of key artifacts (Ops Concept, Requirements, Plans, Design, Code) in each spiral cycle.

- ◆ Avoid premature sequential commitments.

→ Spiral Invariant 2 - *Each Cycle Does Objectives, Constraints, Alternatives, Risks, Review, Commitment to Proceed (STAKEHOLDER PARTICIPATION)*

- ◆ Each cycle considers critical stakeholder objectives and constraints, product and process alternatives, risk identification and resolution, stakeholder review, and commitment to proceed.

- ◆ Avoid commitment to alternatives that are risky or unacceptable to stakeholders.

- ◆ Also avoids wasting effort on unusable alternatives.

→ Spiral Invariant 3 - *Level of Effort Driven by Risk Considerations*

- ◆ Level of effort on each activity within each cycle driven by risk considerations.

- ◆ Avoid too little or too much of each activity.

- ◆ Avoids overkill or belated risk resolution.

→ Spiral Invariant 4 - *Degree of Detail Driven by Risk Considerations (ARTIFACTS CONSIDERATIONS)*

- ◆ Degree of detail of artifacts produced in each cycle driven by risk considerations.

- ◆ Avoids too little or too much of each artifact.

- ◆ Avoids overkill or belated risk resolution.

→ Spiral Invariant 5 - *Use of Anchor Point Milestones: LCO, LCA, IOC (LIFE CYCLE CHECKPOINTS)*

- ◆ Managing stakeholder life cycle commitments via the LCO, LCA, and IOC anchor point milestones.

- ◆ Avoids analysis paralysis, unrealistic expectations, requirements creep, architectural drift, COTS shortfalls or incompatibilities, unsustainable architectures, traumatic cutovers, useless systems

→ *Spiral Invariant 6 - Emphasis on System and Life Cycle Activities and Artifacts (VISIBLE RESULTS)*

- ◆ Emphasis on system and life cycle activities and artifacts rather than software and initial development activities and artifacts.
- ◆ Avoids premature suboptimization on hardware, software, or development considerations

Additionally, we'd like to include a formal definition provided by the author (Boehm, 2000):

The spiral development model is a **risk-driven process model generator**. It is used to guide multi-stakeholder concurrent engineering of software intensive systems. It has two main distinguishing features. One is a **cyclic** approach for incrementally growing a system's degree of definition and implementation while decreasing its degree of risk. The other is a set of **anchor point milestones** for ensuring stakeholder commitment to feasible and mutually satisfactory system solutions.

Keywords: software development methodologies, Spiral Model, Barry Boehm

2. MEETINGS

Boehm does not explicitly mention necessary meetings for the Spiral model in his research. However, we have inferred from his documents which meetings would likely be required in a successful implementation of the Spiral model.

What meetings does this methodology require?

2.1 Product Requirements Meeting

2.1.1 Attendees

- Project Manager, Stakeholders

2.1.2 Agenda

- Welcome
- Discussion of project specifics
- Definition of software requirements
- Creation of Requirements & Lifecycle Plan

2.1.3 Purpose

- The purpose of the Product Requirements Meeting is to identify the specifics of the project to be reached. Working with the stakeholders, project and software requirements will be identified and documented. This is equivalent to a requirements elicitation session. This meeting will result in the creation of the Requirements & Lifecycle Plan.

2.1.4 Event

- Held once at the beginning of the project.

2.2 Phase Planning Meeting

2.2.1 Attendees

- All employees

2.2.2 Agenda

- Welcome
- Declaration of work to be completed
- Assignment of tasks

2.2.3 Purpose

- The purpose of the Phase Planning Meeting is to identify which portions of the project will be completed during the duration of each phase. The Project Manager will declare what work will be completed in each phase, and assign tasks to the appropriate teams and employees.

2.2.4 Event

- Held once at the beginning of each phase.

2.3 Phase Review Meeting

2.2.1 Attendees

- All employees, stakeholders

2.2.2 Agenda

- Welcome
- Declaration of completed work

2.2.3 Purpose

- The purpose of the Phase Review Meeting is to declare to the Project Manager and stakeholders the portions of the project that have been completed during the duration of each phase. Completed work will be discussed and declared, and both stakeholders and the Project Manager will either accept or reject the completed software.

2.2.4 Event

- Held once at the end of each phase.

2.3 Phase Retrospective Meeting

2.3.1 Attendees

- All employees,

2.3.2 Agenda

- Welcome
- Reflection of previous phase
- Planning for future challenges

2.3.3 Purpose

- The purpose of the Phase Retrospective Meeting is to reflect on the previous phase and create solutions for flaws and areas of improvement. Completed work will be discussed and areas of concern will be noted. This meeting allows team members to share their opinions on the previous phase.

2.3.4 Event

- Held once at the conclusion of each phase.

2.4 Weekly Stand-up

2.4.1 Attendees

- All employees

2.4.2 Agenda

- Welcome
- Report of activities

2.4.3 Purpose

- The purpose of the Weekly Stand-up is to allow each team member the opportunity to give a brief report of the previous week's activities, what they will do over the course of the next week, and what obstacles they are facing. This meeting allows the Project Manager to see what work has been completed over the week between different employees and teams.

2.4.4 Event

- Recurring, held once per week.

3. DOCUMENTS

Though Boehm mentions documents to be completed by name, he does not explicitly mention necessary audiences or authors. We have inferred from his research which authors and audiences would likely be required in a successful implementation of the Spiral model. As such, we have marked authors and audiences in red italics.

3.1 Requirements & Lifecycle Plan

3.1.1 Author

- *Project Manager, Software Engineers, Software Testers*

3.1.2 Intended Audience

- *All employees and stakeholders.*

3.1.3 Purpose

- A Requirements & Lifecycle Plan defines how a project team will both manage and fulfill all stakeholder requirements within a project. A proper plan will contain everything from initial project requirements to detailed documentation of necessary functionality. A requirements elicitation session may be necessary for the completion of this document (Jonker & Gomstyn, 2025).

3.1.4 Deadline

- Completion of a Requirements & Lifecycle document should be the first priority of any project. It should be completed immediately upon the project start.

3.2 Development Plan

3.2.1 Author

- *Project Manager, Software Engineers*

3.2.2 Intended Audience

- *All employees and stakeholders.*

3.2.3 Purpose

- The Development Plan will serve as an initial, prototypical design of the project. It will clearly define each component's requirements and give an outline of features to meet these requirements (Purdue, 2024). This plan will be used to create the project and give an outline of all software engineering work to be completed.

3.2.4 Deadline

- Initial completion of the Development Plan document is necessary before coding work can be attempted. However, the document should be updated frequently as requirements and coding decisions change throughout the project. It should be completed immediately following the Requirements & Lifecycle document and updated frequently for the remainder of the project.

3.3 Integration Plan

3.3.1 Author

- *Project Manager, Software Engineers, Software Testers*

3.3.2 Intended Audience

- *Software Engineers.*

3.3.3 Purpose

- This is an integration document. As such, it will create a plan for the different components of the software to work together as intended (Cser, 2023).

3.3.4 Deadline

- Completion of the Integration Plan document is necessary before integration of software components can be attempted. Therefore, it should be completed after development of individual components and before integration begins.

3.4 Integration Plan

3.4.1 Author

- *Project Manager, Software Engineers, Software Testers*

3.4.2 Intended Audience

- *Software Engineers.*

3.4.3 Purpose

- This is an integration document. As such, it will create a plan for the different components of the software to work together as intended (Cser, 2023).

3.4.4 Deadline

- Completion of the Integration Plan document is necessary before integration of software components can be attempted. Therefore, it should be completed after development of individual components and before integration begins.

3.5 Test Plan

3.5.1 Author

- *Project Manager, Software Testers*

3.5.2 Intended Audience

- *Software Testers*

3.5.3 Purpose

All tests created for this project will be organized into a test plan document. This document will be updated daily as new tests are run, completed, and created. This document will serve as a guide for all tests created and executed.

3.5.4 Deadline

- Completion of the Testing Plan document will begin during developmental work as unit tests and developer-side tests are completed. The software testers are responsible to update this document with all tests created and completed upon the software.

4. ROLES

Boehm does not explicitly mention necessary roles for the Spiral model in his research. However, we have inferred from his documents which roles would likely be required in a successful implementation of the Spiral model.

4.1 Project Manager

4.1.1 Qualifications

- Significant experience in development, testing, and management.

4.1.2 Responsibilities

- The project manager should manage both the development of the product and client relations. They should also assist in drafting documentation and designing the finished product. The Project Manager will preside over the entire development process and aim the project towards completion (Ovcharenko, 2024).

4.2 Software Engineer

4.2.1 Qualifications

- College degree in software engineering, computer science, or a related field.

4.2.2 Responsibilities

- The software engineer is responsible for developing the code for the project. They will collaborate with other engineers to create a working product. Additionally, they will assist with creation of documentation, unit testing, and program design.

4.3 Software Tester

4.3.1 Qualifications

- College degree in software engineering, computer science, or a related field.

4.3.2 Responsibilities

- The software tester is responsible for thoroughly testing the code for the project. They will collaborate with other engineers to test the product for bugs, flaws, and security vulnerabilities. They will also assist with creation of testing documentation and test-driven program design.

5. CHECKPOINTS

What milestones and/or checkpoints does this methodology require?

- The Spiral Model enforces structured checkpoints that ensure disciplined progress. For this purpose, checkpoints are classified in two main areas:
 - CYCLE CHECKPOINTS
 - ANCHOR POINT MILESTONES
- The Spiral Model possesses intermediate milestones that serve as commitment points and progress checkpoints. They are the following anchor point milestones: *Life Cycle Objectives* (LCO), *Life Cycle Architecture* (LCA), and *Initial Operational Capability* (IOC).
- These can be described as stakeholder commitment points in the software life cycle:
 - LCO is the stakeholders commitment to support architecting;
 - LCA is the stakeholders' commitment to support full life cycle;
 - And IOC is the stakeholders' commitment to support operations. checkpoints,

5.0 Checkpoint 0

5.0.1 What is the name of the checkpoint?

Cycle Activities Checkpoints.

5.0.2 How does one know that the checkpoint is reached?

- There are a set of activities in each quadrant of *Figure 1* that need to be done in each spiral cycle. These include:
 - Consideration of critical-stakeholder objectives and constraints;

- Elaboration and evaluation of project and process alternatives for achieving the objectives subject to the constraints;
- Identification and resolution of risks attendant on choices of alternative solutions; and,
- Stakeholders' review and commitment to proceed based on satisfaction of their critical objectives and constraints.

If all of these are not considered, the project may be prematurely committed to alternatives that are either unacceptable to key stakeholders or overly risky.

- Following these cycle activities checkpoints, there are main anchor point milestones that serve as lifecycle main points of commitment delivery between the development team and other stakeholders.

5.1 Checkpoint 1 - LCO

5.1.1 What is the name of the checkpoint?

- Life Cycle Objectives (LCO) - "... *getting [romantically] engaged.*"

5.1.2 How does one know that the checkpoint is reached?

- "The focus of the LCO review is to ensure that at least one architecture choice is viable from a business perspective."
- FEASIBILITY RATIONALE: Assurance of consistency among the following elements:

Milestone Element	Description
Definition of Operational Concept	● Top-level system objectives and scope: ○ System boundary ○ Environment parameters and assumptions

Milestone Element	Description
	○ Evolution parameters ● Operational concept: ○ Operations and maintenance scenarios and parameters ○ Organizational life-cycle responsibilities (stakeholders)
System Prototype(s)	● Exercise key usage scenarios. ● Resolve critical risks.
Definition of System Requirements	● Top-level functions, interfaces, quality attribute levels, including: ○ Growth vectors and priorities ○ Prototypes ● Stakeholders' concurrence on essentials
Definition of System & Software Architecture	● Top-level definition of at least one feasible architecture ○ Physical and logical elements and relationships ○ Choices of COTS and reusable software elements ○ Identification of infeasible architecture options
Definition of Life-Cycle Plan	● Identification of life-cycle stakeholders ○ Users, customers, developers, maintainers, interoperators, general public, others ● Identification of life-cycle process model ○ Top-level stages, increments ● Top-level WWWWWHH² by stage

- The previous consistency criteria aligns with the following points:
 - Via analysis, measurement, prototyping, simulation, etc.
 - Business case analysis for requirements, feasible architectures.

5.2 Checkpoint 2 - LCA

5.2.1 What is the name of the checkpoint?

- Life Cycle Architecture (LCA) - "... *getting married*"

² WWWWWHH: Why, What, When, Who, Where, How, How Much

5.2.2 How does one know that the checkpoint is reached?

- “The focus of the LCA review is to commit to a single detailed definition of the review artifacts. [...] The LCA milestone is particularly important, as its pass/fail criteria enable stakeholders to hold up projects attempting to proceed into evolutionary or incremental development without a life cycle architecture.”
- FEASIBILITY RATIONALE: Assurance of consistency among the following elements:

Element	Description
Definition of Operational Concept	• Elaboration of system objectives and scope of increment • Elaboration of operational concept by increment
System Prototype(s)	• Exercise range of usage scenarios • Resolve major outstanding risks
Definition of System Requirements	• Elaboration of functions, interfaces, quality attributes, and prototypes by increment ◦ Identification of TBD's (to-be-determined items) • Stakeholders' concurrence on their priority concerns
Definition of System & Software Architecture	• Choice of architecture and elaboration by increment ◦ Physical and logical components, connectors, configurations, constraints ◦ COTS, reuse choices ◦ Domain-architecture and architectural style choices • Architecture evolution parameters
Definition of Life-Cycle Plan	• Elaboration of WWWWWHH³ for Initial Operational Capability (IOC) ◦ Partial elaboration, identification of key TBDs for later increments

- All major risks resolved OR covered by a risk management plan.

³ WWWWWHH: Why, What, When, Who, Where, How, How Much

5.3 Checkpoint 3 - IOC

5.3.1 What is the name of the checkpoint?

- Initial Operational Capability (IOC) - “... *having your first child.*”

5.3.2 How does one know that the checkpoint is reached?

- The Initial Operational capability (IOC) is the first the users will see of a functioning system.
- The key elements of the IOC milestone are:
 - *Software preparation*, including both operational and support software with appropriate commentary and documentation; data preparation or conversion; the necessary licenses and rights for COTS and reused software, and appropriate operational readiness testing.
 - *Site preparation*, including facilities, equipment, supplies, and COTS vendor support arrangements.
 - *User, operator and maintainer preparation*, including selection, teambuilding, training and other qualification for familiarization, usage, operations, or maintenance.

7. REFERENCES

7.1 Primary Sources

- Boehm, B.W. (1988) 'A Spiral Model of Software Development and Enhancement', *Computer*, 21(5), pp. 61–72. Available at: <<https://ieeexplore.ieee.org/document/59/>> (Accessed: 04 April 2025).
- Boehm, B.W. (2000) *Spiral Development: Experience, Principles, and Refinements*, SEI Digital Library. Available at: <<https://insights.sei.cmu.edu/library/spiral-development-experience-principles-and-refinements-spiral-development-workshop-february-9-2000/>> (Accessed: 04 April 2025).

7.2 Secondary Sources

- Cser, T. 2023, Checkpoints, *Functionize*, <<https://www.functionize.com/automated-testing/checkpoints>> (Accessed: 7 Apr 2025).
 - Tamas Cser is an expert in the realm of software engineering, project planning, and software testing. He is the founder and CEO of Functionize, a test-automation provider. Due to his years of experience as a project manager and team lead, Cser is a reliable source for information about project management and software testing.
- Jonker, A, & Gomstyn A (2025) 'What is requirements management?', *IBM*, Available at: <[https://www.ibm.com/think/topics/what-is-requirements-management#:~:text=A%20requirements%20management%20plan%20\(RMP\)%20helps%20explain%20how%20a%20project,market%20and%20improve%20quality%20control](https://www.ibm.com/think/topics/what-is-requirements-management#:~:text=A%20requirements%20management%20plan%20(RMP)%20helps%20explain%20how%20a%20project,market%20and%20improve%20quality%20control)> (Accessed 7 April 2025).
 - Alexandra Jonker and Alice Gomstyn are both experienced authors at IBM. IBM is one of the nation's oldest and most reliable innovators in computer science and computer products. Jonker and Gomstyn have long worked with IBM to create

informative, factually-based information pieces in software engineering and computer science. Therefore, they are a reliable source.

- Purdue n.d., Stage Two: Preliminary Design, Purdue University,
<https://owl.purdue.edu/owl/subject_specific_writing/writing_in_engineering/engineering_project_documentation/stage_two_preliminary_design.html> (Accessed 7 Apr 2025).
 - This information comes directly from Purdue University, one of the nation's most reliable and revered research institutions. It should go without explanation that Purdue University is a reliable source of information.
- Ovcharenko, D. 2024. Perfect Software Development Team: Roles and Responsibilities, Alcor,
<<https://alcor-bpo.com/10-key-roles-in-a-software-development-team-who-is-responsible-for-what/>> (Accessed 7 Apr 2025).
 - Dmytro Ovcharenko is the founder and CEO of Alcor, a company centered on assisting tech companies build teams and manage projects. Ovcharenko has several years of deep experience as a project manager and team lead over several software engineering projects. Therefore, he is a reliable source of information in the project management field of software engineering.